

7. REST 서비스 사용하기

API: 데이터 받거나 제공.

- RestTemplate : Spring framework에서 제공하는 간단하고 동기화된 클라이언트
- Traverson : Spring HATEOAS에서 제공하는 하이퍼 링크를 인식하는 동기화 REST 클라이언트로 같은 이름의 자바스크립트 라이브러리로서의 버전.
- WebClient : Spring 5에서 소개된 비동기 비동기 REST 클라이언트.

7.1 RestTemplate으로 REST 엔드포인트 사용.

- Client => Client Instance와 요청 객체 생성, 해당 요청 실행, 응답 분석하여 관련 도메인 객체와 연관시켜 처리. + 예기치.

L 작업이 전부함 + 반복적. => 'RestTemplate'로 지랄 수 있음.

- RestTemplate => REST Resource 사용하는데 빈잡한 일 처리.
- RestTemplate이 정의하는 중요한 작업 수행하는 12개의 메소드.

1) GET

- L getObject() : GET 요청 후, 응답을 객체로 바로 반환 (응답 Body만 필요시)
- L getEntity() : GET 요청 후, 응답(ResponseEntity) 반환 (헤더 + 상태코드 포함 필요시)

2) POST

- L postForObject() : POST 요청 후 응답을 객체로 반환 (form 전송 or JSON 전송에 사용)
- L postForEntity() : POST 요청 후, 전체 응답 반환 (상태코드 확인이 필요한 경우.)

3) 생성 후 URI 반환

- L postForLocation() : POST 요청 후, Location 헤더에 있는 URI 반환 (자원 생성 시 사용)

4) 수정 (PUT)

L put() : PUT 요청으로 자천 업데이트 ... 응답x

5) 삭제 (DELETE)

L delete() : DELETE 요청으로 자천 삭제 ... 응답x.

6) URI 조작

L headForHeaders() : HEAD 요청 보내고 헤더 정보만 반환
보통 유효성 검사에 사용

7) OPTIONS 요청

L optionsForAllow() : OPTIONS 요청 보내고 가능한 HTTP 메소드 반환

8) URI 쿼리

L getForLocation() : GET 요청 후 Location 헤더 반환

9) 일반 요청

L exchange() : 모든 HTTP 요청을 처리할 수 있는 범용 메소드
GET, POST, PUT 등 지정 가능

10) 요청-응답 템플릿

L execute() : 가장 자주 요청 실행 메소드.
클백을 통해 요청/응답 커스터마이징.

1.1.1 리소스 가져오기 (GET)

-> `getForObject()`, `getForEntity()`

1) API 코너 데이터 가져오기.

HATEOAS가 활성화되지 않았다면? -> `getForObject()` 사용.

ex)

```
public Ingredient getIngredientById(String ingredientId) {  
    return rest.getForObject("http://localhost:8080/.../{id}",  
                             Ingredient.class, ingredientId);  
}
```

-> `getForObject(String url, Class<T> responseType, Object... uriVariables)`

- 여기서는 URL 변수의 값인 리스트와 URL 문자열을 연자로 함께 제공된 `getForObject()` 사용
- `ingredientId`는 `{id}`에 들어감. 변수 매개변수들은 주어진 순서대로 지정.
- 두 번째 매개변수 : 응답이 바인딩되는 타입.
- JSON 형식의 응답 데이터가 객체로 역직렬화되어 반환.

Map 사용해서 URL 변수 지정 가능

ex)

```
Map<String, String> params = new HashMap<>();  
params.put("id", ingredientId);  
return rest.getForObject("http://localhost:8080/.../{id}",  
                          Ingredient.class, params);  
}
```

↑
키는 "id"이며 요청이 수행될 때
{id}에 key가 id인 Map 항목 값으로 교체.

- URI 기계번수 사용할 때는 URI 객체 생성하여 getForObject 드림.

ex)

:

URI uri = UriComponentsBuilder

.fromHttpUrl("http:// ~ / {id}") ... ①

.buildAndExpand(params); ... ②

```
return rest.getForObject(uri, Ingredient.class);
}
```

① fromHttpUrl(String uriTemplate)

: URI 템플릿을 문자열로부터 시작해 URI 조립 시작.

- 역할 : URI 빌더 기호 쿼리 지정.
- 입력 : http://api.com/users/{id} 같이 {} 템플릿 가능.
- 반드시 http:// or https:// 형식 포함해야 함.

② buildAndExpand

: {id} 같은 URI 경로 템플릿이 실제 값 채움.

- 역할 : {id} → 'Value'로 ^{여기서} 변환하여 전체 URI를 만든다.
- 입력 : Map<String, String> 형식으로 {변수명 = 값} 구조 넘김.
- build().expand(value) or build().expand("id", value) 가능.

③ encode()

: URI 전체를 RFC 3986에 따라 인코딩.

- 목적
 - 공백 → %20
 - 한글 ⇒ %EC%95%88%EB%85%95
 - 특수문자 ⇒ 안전하게 변환

- 권장: **encode()**는 **buildAndExpand()** 이후에 사용하는 것이 맞다.

④ toUri()

: 최종적으로 java.net.URI 객체로 변환.

- 사용처: RestTemplate, WebClient, RedirectView, HttpHeaders.setLocation() 사용가능

But, Client 가 이미 이 쿼리 결과로 필요한 것이 있다면 `getForEntity()` 사용.

`getForEntity()` → 도메인 객체 포함하는 `ResponseEntity` 객체 반환.

ex)

```
public Ingredient getIngredientById (String ingredientId) {
```

```
    rest.getForEntity (" http://localhost/ ... / {id} ",
```

```
        Ingredient.class, ingredientId);
```

```
    log.info (" Fetched time " +  
        responseEntity.getHeader().getDate());
```

```
    return responseEntity.getBody();
```

응답 Date 헤더 확인하고 쓸 때.

```
}
```

`response.getStatusCode()` : HTTP 상태코드

`response.getBody()` : 실제 데이터

`response.getHeaders()` : HTTP 응답 헤더 전체.

7.1.2 리소스 쓰기 (PUT)

- void put (String url, Object request, Object... uriVariables)

L REST API에 PUT 요청 보낸다.

L 서버에 지정된 자천 덮어쓰이거나 새로 생성

L 응답 값 필요없는 경우가 사용.

ex)

⋮

```
rest.put ("http://localhost:8080/ ... / {id} ",
```

```
    ingredient,
```

```
    ingredient.getId());
```

⋮

7.1.3 리소스 삭제하기 (DELETE)

- void delete (String url, Object... uriVariables)

L 지정된 URI로 HTTP DELETE 요청 전송

L return 값 x

L 서버에서 특정 자원 제거할 때 사용.

ex)

```
rest.delete("http://.../id",  
            ingredient.getId());
```

7.1.4 리소스 데이터 추가하기 (POST)

- postForObject() : POST 요청 후, 응답 body만 반환 ... 객체 반환

- postForEntity() : post 요청 후, 전체 응답 반환 ... ResponseEntity<T> 반환

- postForLocation() : POST 요청 후, Location 헤더 반환 ... URI 반환

1) postForObject()

=> 서버로 데이터 전송, 응답으로 생성된 객체의 정보만 받으면 편리할 때.

ex)

```
rest.postForObject("http://localhost:8080/...",  
                  ingredient, ...> 전송될 객체 ...> URI  
                  Ingredient.class);  
                  ...> 객체 타입.
```

2) postForEntity()

=> 상태코드 (201 Created, 400 Bad Request) 확인이 필요할 때.

=> Header 정보 사용 or Redirect 처리 확인 필요할 때.

ex)

⋮

```
ResponseEntity<Ingredient> response = rest.postForEntity (
```

```
    "http:// ~ ",
```

```
    ingredient,
```

```
    Ingredient.class );
```

⋮

3) postForLocation()

=> 서버가 Location 헤더 통해 생성된 자원의 URI 알려주는 경우

=> ex) Location : https://api.example.com/user/123

ex)

```
public URI _____ (Ingredient ingredient) {
```

```
    return rest.postForLocation ("http:// ~ ",
```

```
        ingredient );
```